CLOUD & DEVOPS

Serverless Visual Guide

Functions as a service —
events in, scale-out, pay per invocation.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Execution Model

You upload code; the platform runs it per event.

No servers to provision, patch, or watch. Scale is the platform's problem.

- Stateless by design: persist to S3/DynamoDB/RDS — never local disk.
- Each invocation is isolated; concurrency = parallel executions, not threads.
- 15-minute max execution: split long work into steps or use containers.

STEP 2/6

Event Sources: What Triggers Functions

Push and pull — the function just reacts.

API Gateway

HTTP REST/WebSocket → request/response

S3 / Storage

object created → thumbnails, transcoding

SQS / Queues

message batch → workers that scale with backlog

Schedules

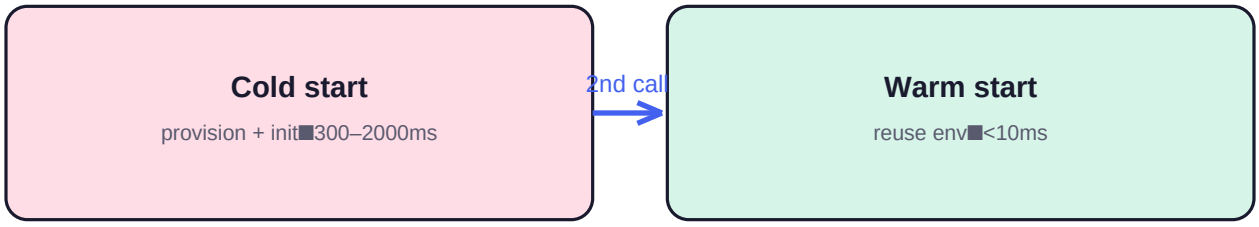
EventBridge cron → nightly reports, cleanup

Queue triggers give free backpressure: messages wait safely while concurrency catches up.

STEP 3/6

Cold Starts & Mitigation

First invocation after idle pays an init tax.



- Keep init light: lazy-load heavy SDKs, slim dependencies, small packages.
- Provisioned concurrency: pre-warmed environments for latency-critical paths (costs more).
- SnapStart / tiered compile: restore from snapshot instead of booting fresh.

STEP 4/6

Limits You Must Design Around

Quotas are guardrails — know them before prod.

Timeout

15 min max — chunk long work

Memory

128MB–10GB — also scales CPU proportionally

Concurrency

default 1000/region — reserve per function

Payload

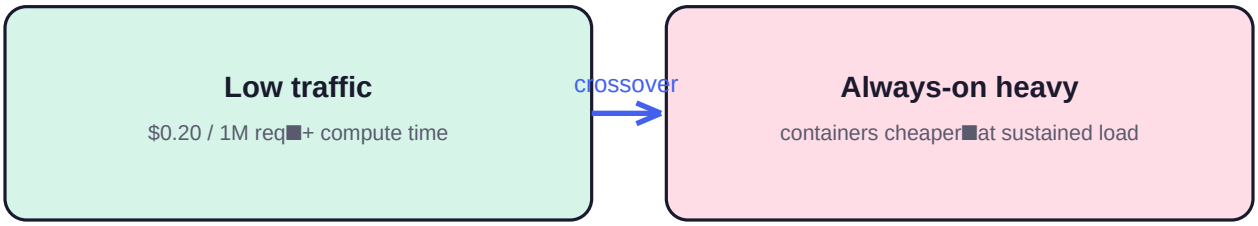
6MB sync / 256KB async — big data via S3, not events

Concurrency is a shared pool: one runaway function can starve the rest — set reserved concurrency per critical fn.

STEP 5/6

The Cost Model

Pay per invocation + GB-seconds. Cheap at idle, watch the tail.



Sweet spot: spiky, bursty, idle-mostly workloads (webhooks, cron, event processing).

- Free tier covers small apps entirely — serverless is the cheapest way to start.
- Cost per request is flat: 10x traffic ≈ 10x bill. Model it before launch.
- Recursive loops (fn triggers itself via S3) can burn money in minutes — add circuit breakers.

STEP 6/6

Serverless Patterns

Common shapes that fit the model well.

API backend

API Gateway + functions + DynamoDB — no servers at all

Async workers

queue → functions scale with backlog, retries built in

Event pipelines

S3 → fn → S3: thumbnails, ETL, virus scan

Glue & cron

scheduled fns: cleanup, reports, sync jobs

STEP ✓/✓

Key Takeaways

Serverless — Remember

- Event in → handler runs → platform scales: zero server management.
- Event sources: HTTP, storage, queues (free backpressure), schedules.
- Cold starts: slim init, provisioned concurrency for hot paths.
- Limits: 15-min timeout, concurrency pools, payload caps — design around them.
- Pay per use: unbeatable for spiky workloads, model cost before scale.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev